

MethylPipeline Overview

Architecture, workflows, and package responsibilities

OpenAI Codex

2026-04-03

Table of contents

1	Executive Summary	1
2	What The Repository Contains	2
2.1	Core Statistical Path	2
2.2	Downstream Interpretation And Support Packages	2
3	Package Architecture	2
4	Shared Data Model	2
5	Main Workflows	3
5.1	Workflow 1: Model Creation	3
5.1.1	Workflow 1 Diagram	4
5.2	Workflow 2: Frozen-Model Prediction	4
5.2.1	Workflow 2 Diagram	5
6	Statistical Character	5
7	Typical Outputs	5
8	Rendering Notes	6

1 Executive Summary

MethylPipeline is a monorepo for DNA methylation analysis. Its core supervised path centers on empirical cumulative distribution functions (ECDFs) and histogram-backed centroid summaries rather than forcing the full workflow into a single parametric family. Around that core, the repository adds packages for clustering, gene mapping, enrichment, prediction, repeated validation, and alignment quality control.

At a practical level, the repository supports two major modes of work:

1. building a stable production model from repeated train-validation splits, and

2. applying a frozen model for predictor-only evaluation.

The codebase is organized as cooperating packages with `methyutils` as the shared mathematical and utility layer.

2 What The Repository Contains

2.1 Core Statistical Path

The central supervised pipeline is:

1. `methylcentroid` builds cohort summaries from per-sample methylation values.
2. `methyldetector` screens loci for differential methylation and exports classifier-ready DMP panels.
3. `methylclassifier` fits or applies classification logic over the detected loci.
4. `methylpredictor` evaluates trained models on holdout or blind samples.
5. `methylvalidation` orchestrates repeated validation, stability analysis, production freeze, and final model creation.

This path is the most tightly coupled statistical spine of the repository.

2.2 Downstream Interpretation And Support Packages

- `methylmapper` maps DMPs to genes and genomic regions.
- `methylenricher` performs enrichment analysis, graph clustering, and module ranking.
- `methylalignmentqc` parses alignment/QC outputs from upstream sequencing workflows.
- `methylcluster` provides exploratory clustering utilities outside the main production path.
- `methyutils` provides shared data structures, ECDF reconstruction, overlap metrics, tests, logging, configuration utilities, and classifier support code.

3 Package Architecture

The package graph is easier to understand as a layered system: shared utilities at the bottom, the supervised methylation workflow in the center, and biological interpretation or QC around it.

4 Shared Data Model

At its lowest common layer, the code works with methylated and unmethylated counts at genomic loci. For sample s and position i ,

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

where m_{si} is the methylated count and u_{si} is the unmethylated count.

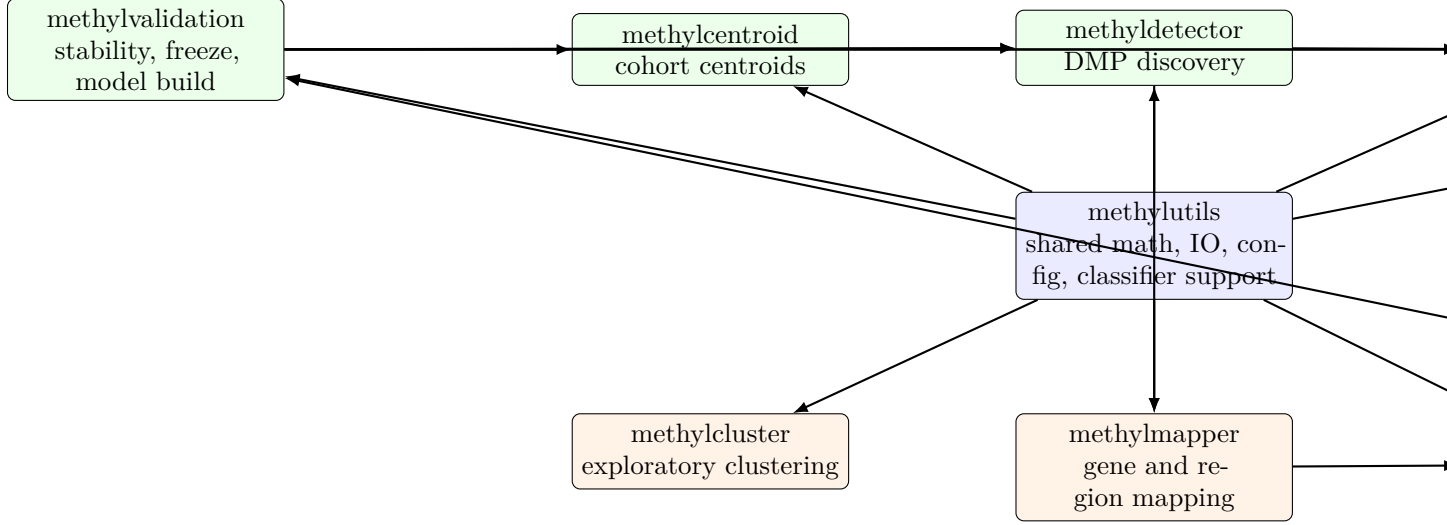


Figure 1: High-level package architecture of MethylPipeline.

Centroid-building packages summarize cohorts through per-locus sufficient statistics:

$$N_i = \sum_s 1, \quad S_{x,i} = \sum_s x_{si}, \quad S_{x^2,i} = \sum_s x_{si}^2.$$

These support the implemented cohort mean and variance estimates:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right).$$

In the ECDF-centered path, each locus also stores histogram counts over fixed methylation bins. Those histograms are used to reconstruct empirical distribution views, approximate overlap metrics, and rank-based or distribution-based comparisons at scale.

5 Main Workflows

5.1 Workflow 1: Model Creation

Model creation is the longer production workflow. It is intentionally separated into three stages:

1. **--stability**: repeated train-validation splits that rerun centroid and detector on the training subset only.
2. **--freeze**: rerun the stable panel on all samples and produce downstream biological interpretation.
3. **--model**: train the final deployable classifier and predictor artifacts using the frozen production project.

This split matters statistically because it reduces leakage between feature discovery and final model construction, even though it is still an internal workflow rather than external validation on an independent cohort.

5.1.1 Workflow 1 Diagram

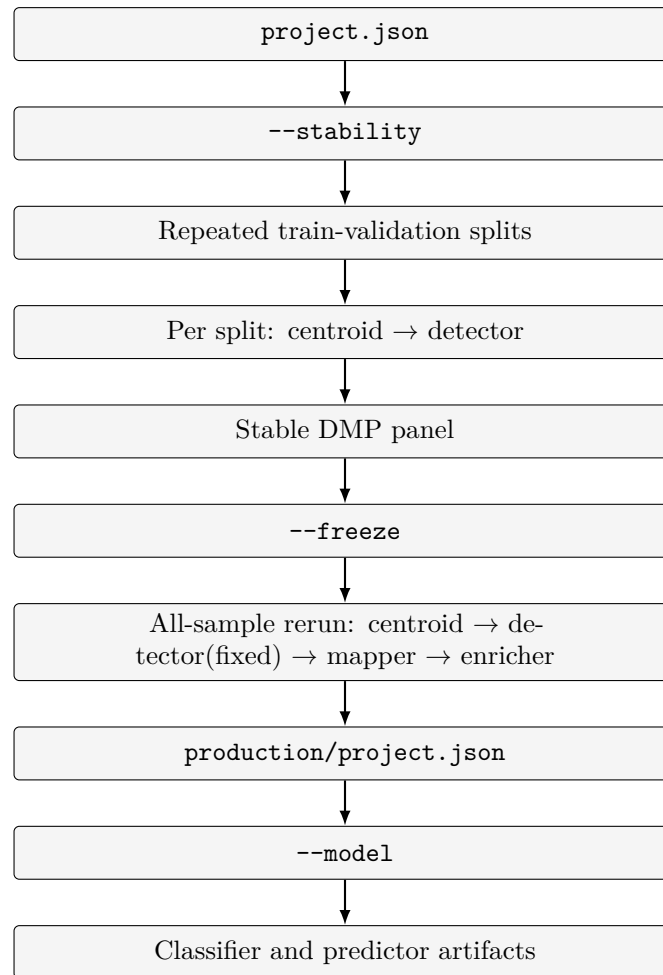


Figure 2: Workflow 1 for creating a stable production model.

5.2 Workflow 2: Frozen-Model Prediction

The second workflow assumes a frozen production project already exists. It runs predictor-only evaluation across repeated splits while reusing the frozen model artifacts.

This workflow answers a different question from Workflow 1. Instead of re-discovering loci and re-training the pipeline for each split, it asks how stable the frozen model's prediction metrics are across sampled validation subsets.

5.2.1 Workflow 2 Diagram

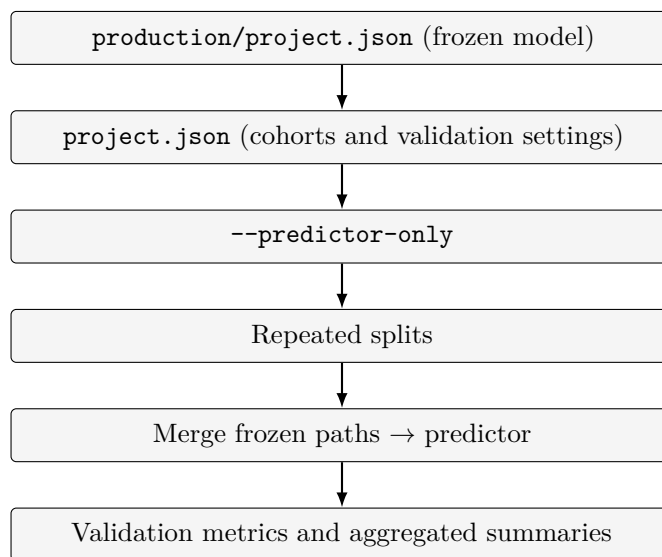


Figure 3: Workflow 2 for predictor-only evaluation of a frozen model.

6 Statistical Character

MethylPipeline mixes several methodological categories, and the documentation should keep them distinct:

- principled methods such as standard hypothesis tests, multiple-testing correction, and established classifiers,
- approximate methods where empirical distributions are reconstructed from summaries or discretized histograms,
- heuristic rules for ranking, thresholding, or aggregation,
- external-service-backed steps in downstream interpretation layers.

That distinction is important because the repository is strongest when it speaks precisely about what is implemented. The core centroid-detector-classifier path is grounded in empirical-distribution summaries, while downstream interpretation layers may depend on graph heuristics, curated databases, or external enrichment services.

7 Typical Outputs

Common outputs across the repository include:

- centroid files summarizing cohort-level methylation behavior,
- detector outputs containing DMP statistics and filtered panels,
- classifier artifacts and predictor metrics,
- stability summaries across Monte Carlo iterations,

- mapping and enrichment outputs for downstream biological interpretation,
- QC summaries extracted from external alignment reports.

A typical production-oriented run produces a `monte_carlo_runs/production/` tree containing the frozen project, stable panel outputs, downstream interpretation products, and final deployable model artifacts.

8 Rendering Notes

This file is intended to render directly with Quarto on a machine that has LaTeX support available. The PDF format is configured to use `lualatex`, and the TikZ diagrams are injected through `include-in-header`.

Example commands:

```
quarto render docs/MethylPipeline-overview.qmd --to html
quarto render docs/MethylPipeline-overview.qmd --to pdf
```

If your Quarto installation already has TinyTeX, TeX Live, or MiKTeX available, the PDF build should render the TikZ diagrams without extra changes.